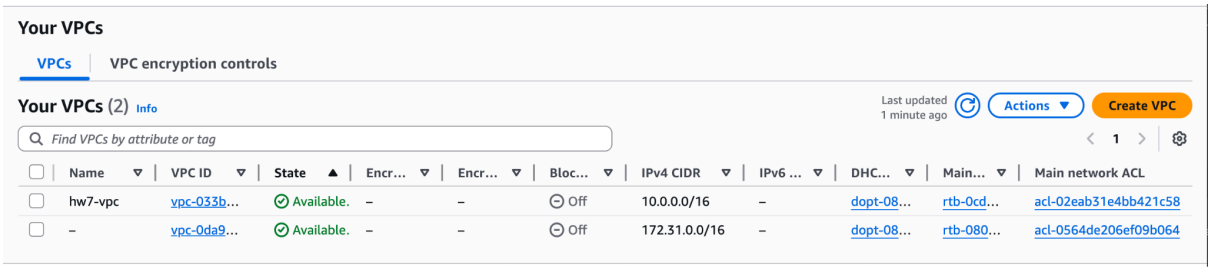
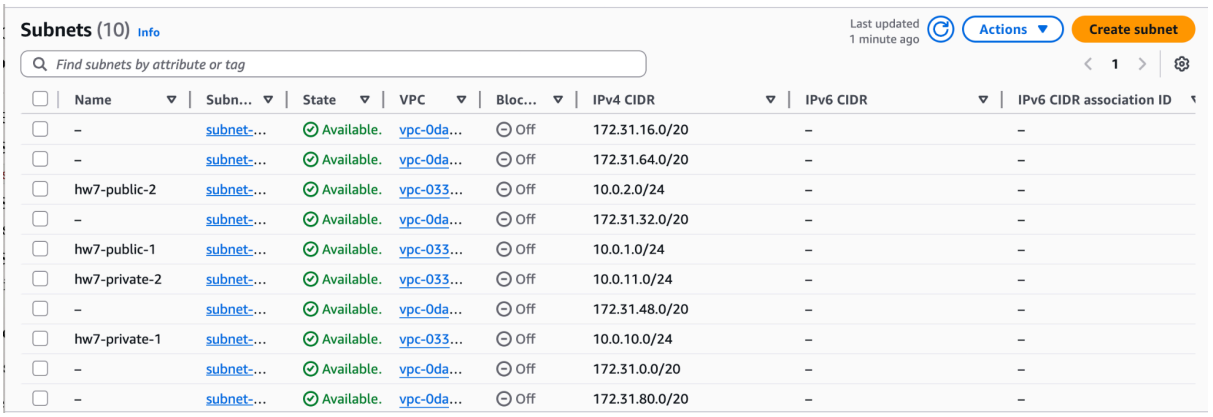
PART - 2

Infrastructure

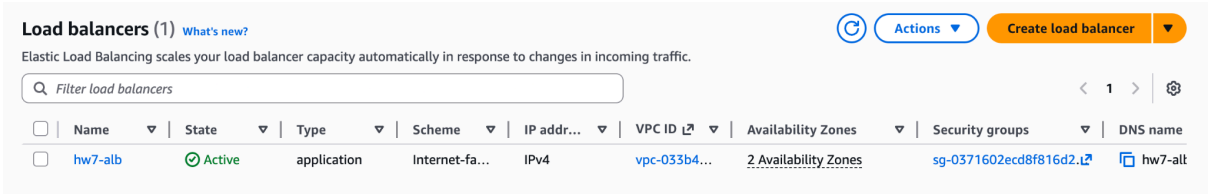
VPC:



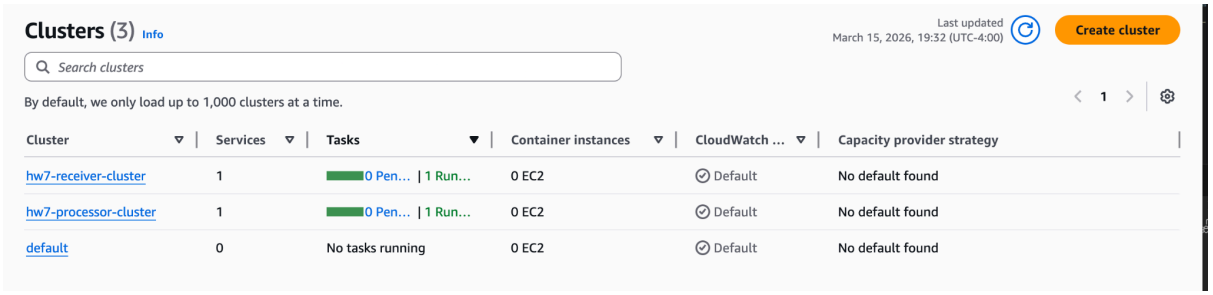
Subnets:



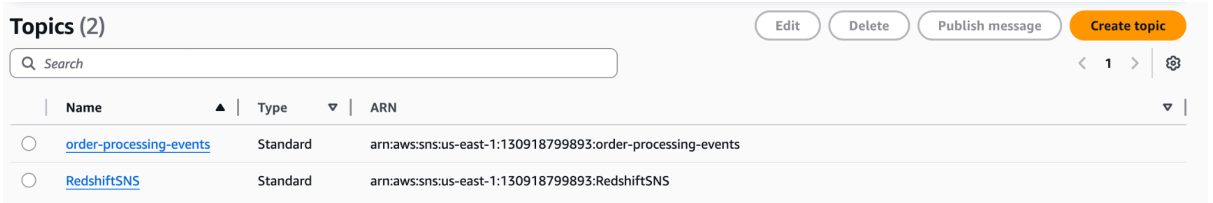
ALB:



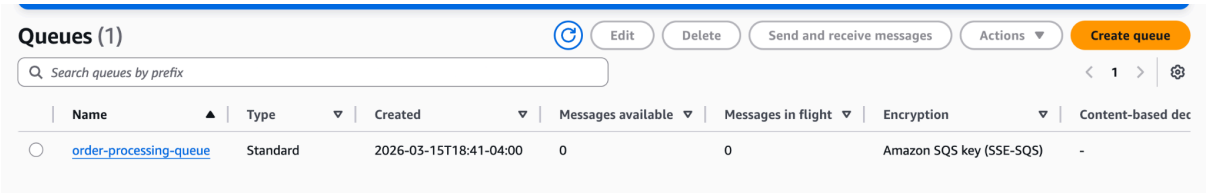
ECS:



SNS:



SQS:



2. Application: Go service with sync and async endpoints

2.1. Buffered channel bottleneck (order-receiver):

```
44
45  // Buffered channel to simulate the payment processor bottleneck.
46  // Capacity of 1 means only 1 payment can be processed at a time,
47  // and each takes 3 seconds → max ~0.33 orders/sec throughput.
48  var paymentSem = make(chan struct{}, 1)
49
50  // simulatePayment blocks the goroutine for 3 seconds using the
51  // buffered channel approach recommended in the assignment.
52  func simulatePayment() {
53      paymentSem <- struct{}{} // acquire slot (blocks if full)
54      time.Sleep(3 * time.Second)
55      <-paymentSem // release slot
56  }
57
```

Testing

this simulates a real payment processor that can only handle one transaction at a time, using the buffered channel approach.

2.2. Sync endpoint (order-receiver):

```go

func syncOrderHandler(c *gin.Context) {
    var req OrderRequest
    if err := c.ShouldBindJSON(&req); err != nil {
        c.JSON(http.StatusBadRequest, gin.H{"error": "Invalid request body"})
        return
    }

    order := Order{
        OrderID:    uuid.New().String(),
        CustomerID: req.CustomerID,
        Status:     "processing",
        Items:      req.Items,
        CreatedAt:  time.Now(),
    }

    log.Printf("[SYNC] Processing order %s for customer %d", order.OrderID, order.CustomerID)

    // Simulate 3-second payment verification (blocking via buffered channel)
    simulatePayment()

    order.Status = "completed"
    log.Printf("[SYNC] Order %s completed", order.OrderID)

    c.JSON(http.StatusOK, order)
}

```

the customer waits the full 3 seconds before getting a response.

2.3Async endpoint (order-receiver):

```go

func asyncOrderHandler(c *gin.Context) {
    var req OrderRequest
    if err := c.ShouldBindJSON(&req); err != nil {
        c.JSON(http.StatusBadRequest, gin.H{"error": "Invalid request body"})
        return
    }

    order := Order{
        OrderID:    uuid.New().String(),
        CustomerID: req.CustomerID,
        Status:     "pending",
        Items:      req.Items,
        CreatedAt:  time.Now(),
    }

    // Publish to SNS
    orderJSON, err := json.Marshal(order)
    if err != nil {
        c.JSON(http.StatusInternalServerError, gin.H{"error": "Failed to marshal order"})
        return
    }

    _, err = snsClient.Publish(&sns.PublishInput{
        TopicArn: aws.String(snsTopicARN),
        Message:  aws.String(string(orderJSON)),
    })
    if err != nil {
        log.Printf("[ASYNC] Failed to publish order %s: %v", order.OrderID, err)
        c.JSON(http.StatusInternalServerError, gin.H{"error": "Failed to queue order"})
        return
    }

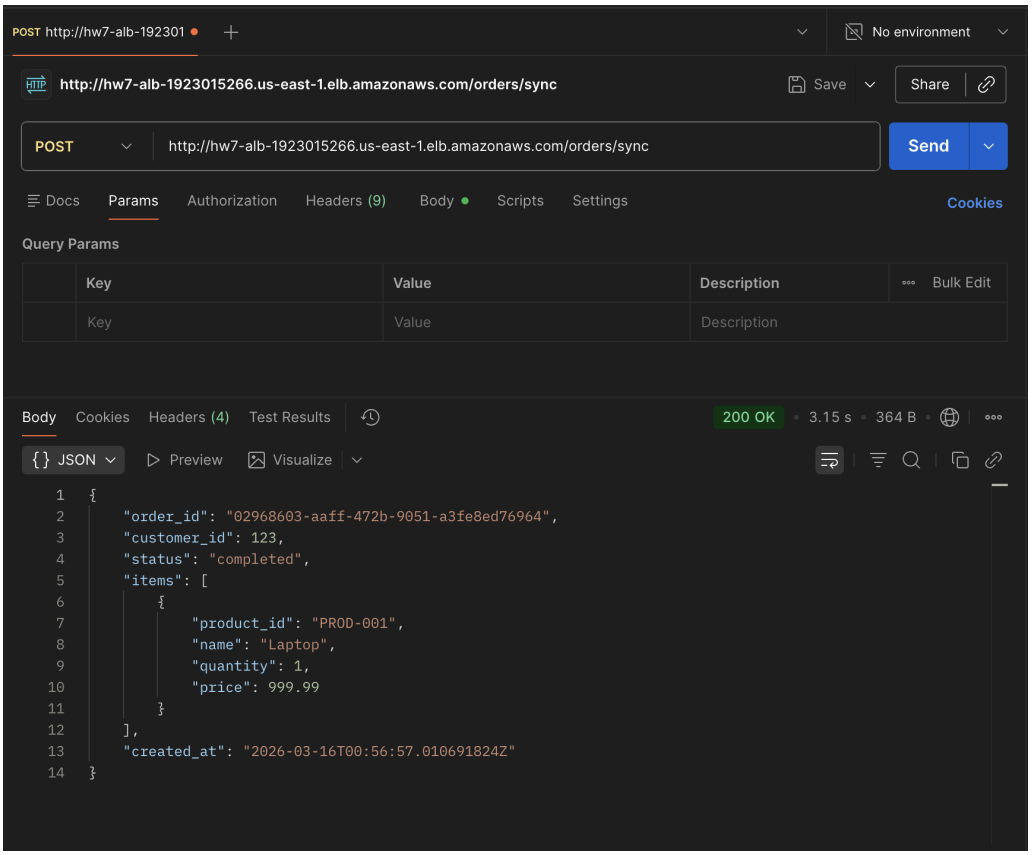
    log.Printf("[ASYNC] Order %s accepted and queued", order.OrderID)

    c.JSON(http.StatusAccepted, order)
}

```

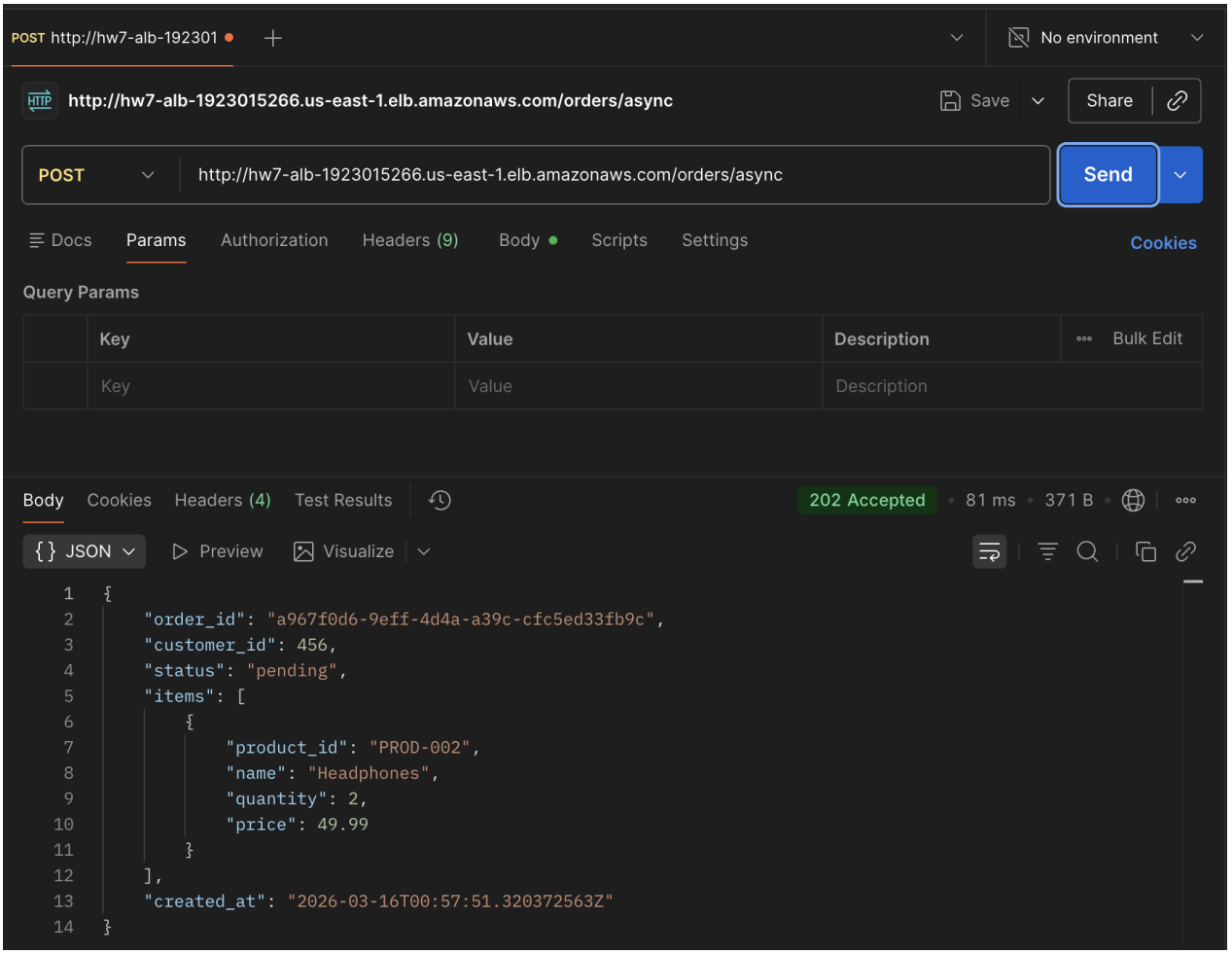
the order is published to SNS and the customer gets an immediate 202 Accepted response. No waiting for payment.

Sync endpoint:



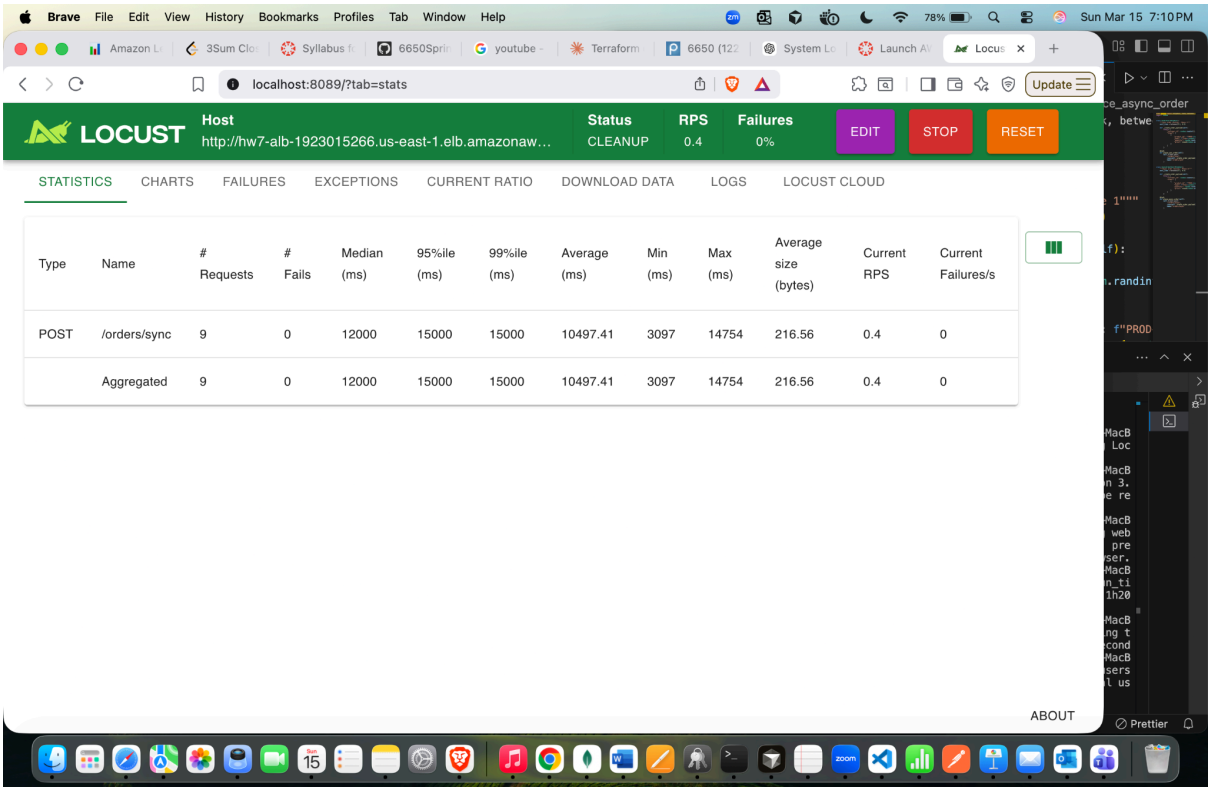
take ~3 seconds and return status `"completed"`.

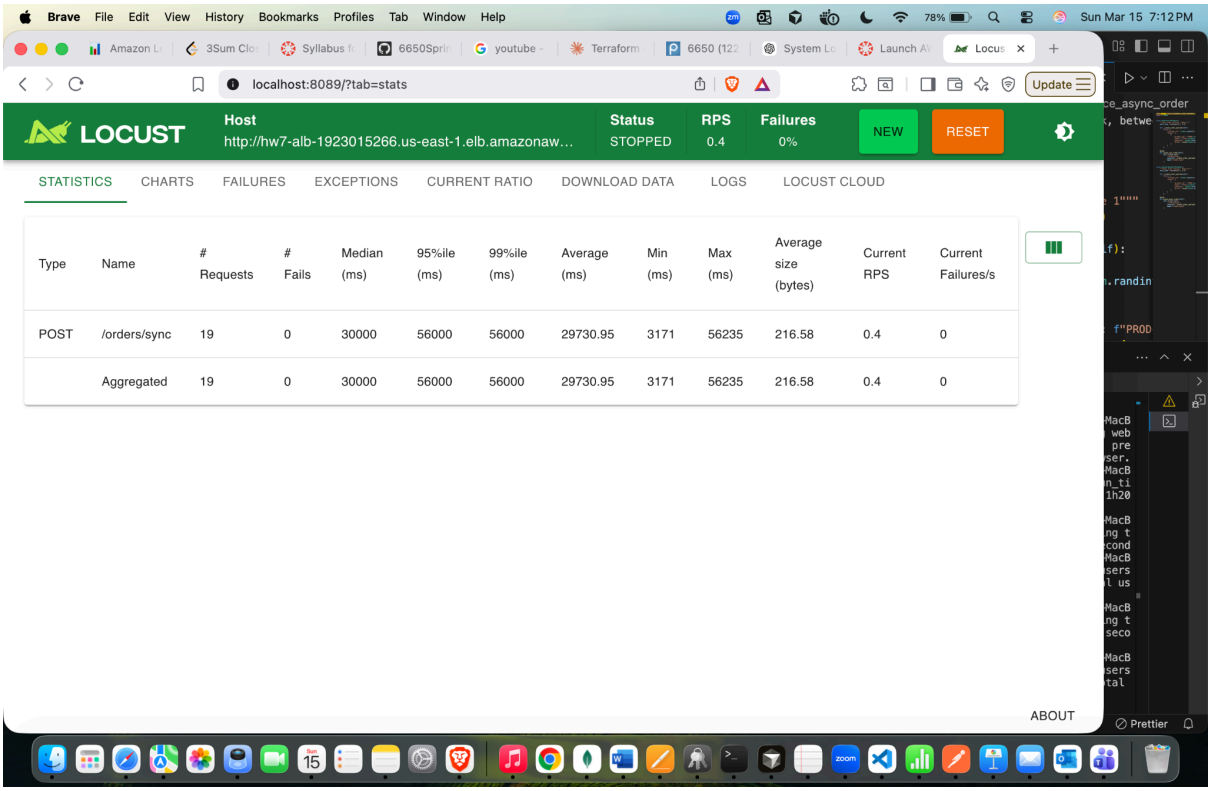
Async endpoint:



return instantly with status `"pending"` and HTTP 202 Accepted. The order is queued in SNS/SQS for background processing.

Phase 1: Sync Endpoint





Two load tests were run using Locust with a wait time of 100-500ms between requests.

Metric	Normal (5 users, 30s)	Flash Sale (20 users, 60s)
Total Requests	9	19
Failures	0 (0%)	0 (0%)
Avg Response Time	10,497 ms	29,731 ms
Median	12,000 ms	30,000 ms
P95	15,000 ms	56,000 ms
Max	14,754 ms	56,235 ms
Throughput (RPS)	0.33	0.33

Observation:

throughput is 0.33 req/s regardless of load.
The server does not crash or return errors it simply makes customers wait.
With 20 concurrent users, the median wait is 30 seconds and worst case nearly a minute.
In a real flash sale, customers would abandon their carts long before receiving a response.

Phase 2: Bottleneck Analysis

Payment processor speed: 1 order per 3 seconds = 0.33 orders/second
With 20 concurrent customers: Maximum throughput = **0.33 orders/second**
Flash sale demand: ~20 orders/second
Orders lost per second: 20 - 0.33 = **~19.67 orders/second**

Metric	Value
Payment processor speed	1 order per 3 seconds = 0.33 orders/second
Maximum throughput (with 20 concurrent customers)	0.33 orders/second
Flash sale demand	~20 orders/second
Orders lost per second	~19.67 orders/second

The bottleneck is the payment processor, not the server.
we cannot make payment processing faster, but you can decouple the customer-facing response from the payment step.
Accept orders immediately, queue them for background processing, and let workers handle payment verification asynchronously.

Phase 3: The Async Solution

Instead of blocking customers for 3 seconds, the **/orders/async** endpoint publishes the order to an SNS topic and immediately returns a 202 Accepted response. SNS fans out to an SQS queue, where background worker goroutines poll and process orders.

AWS resources used:

- SNS Topic: `order-processing-events`
- SQS Queue: `order-processing-queue` (visibility timeout 30s, retention 4 days, long polling 20s)
- Order Receiver: ECS Fargate service behind ALB (handles both `/sync` and `/async`)
- Order Processor: ECS Fargate service polling SQS

Flash sale test results (20 users, 60s):

Metric	Sync (Phase 1)	Async (Phase 3)
Total Requests	19	3,448
Avg Response Time	29,731 ms	46 ms
Median	30,000 ms	42 ms
P95	56,000 ms	68 ms
Throughput (RPS)	0.33	59.5
Failures	0%	0%

Customers receive immediate confirmation while orders are processed in the background

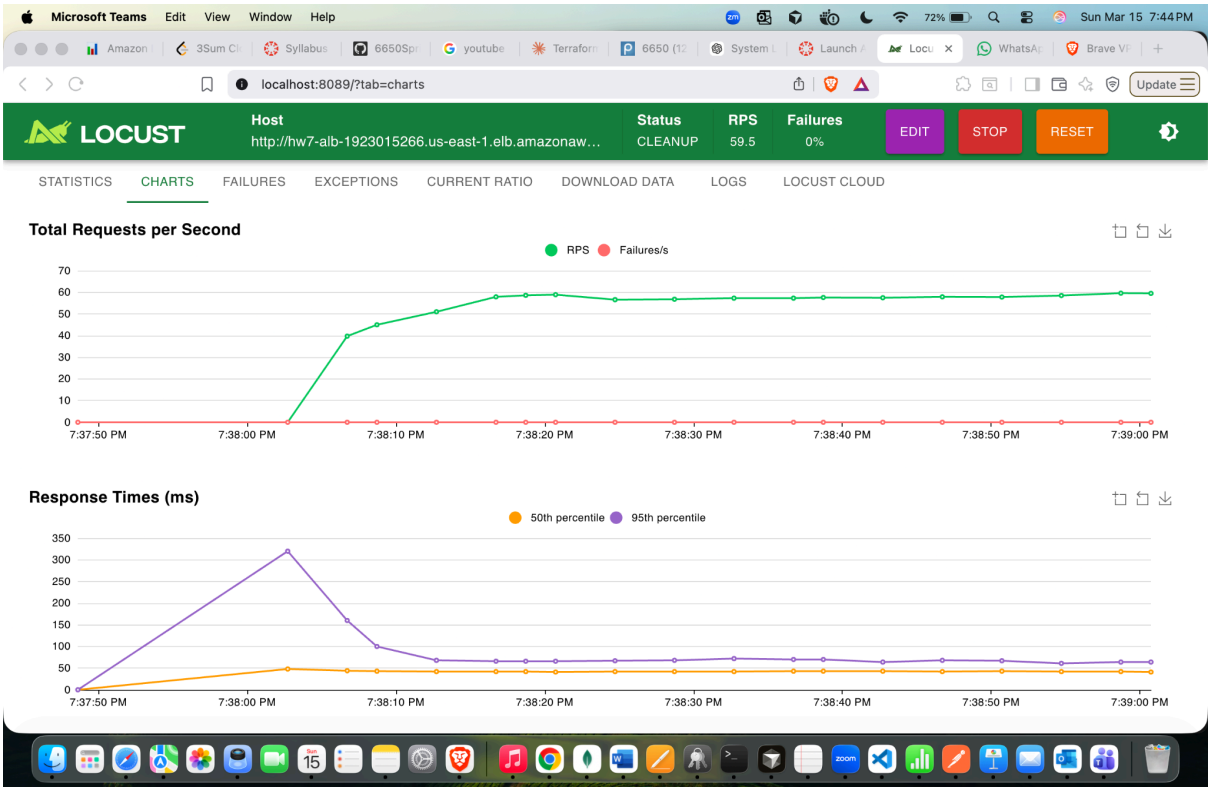
LOCUST

Host: http://hw7-alb-1923015266.us-east-1.elb.amazonaw…
Status: CLEANUP
RPS: 59.5
Failures: 0%

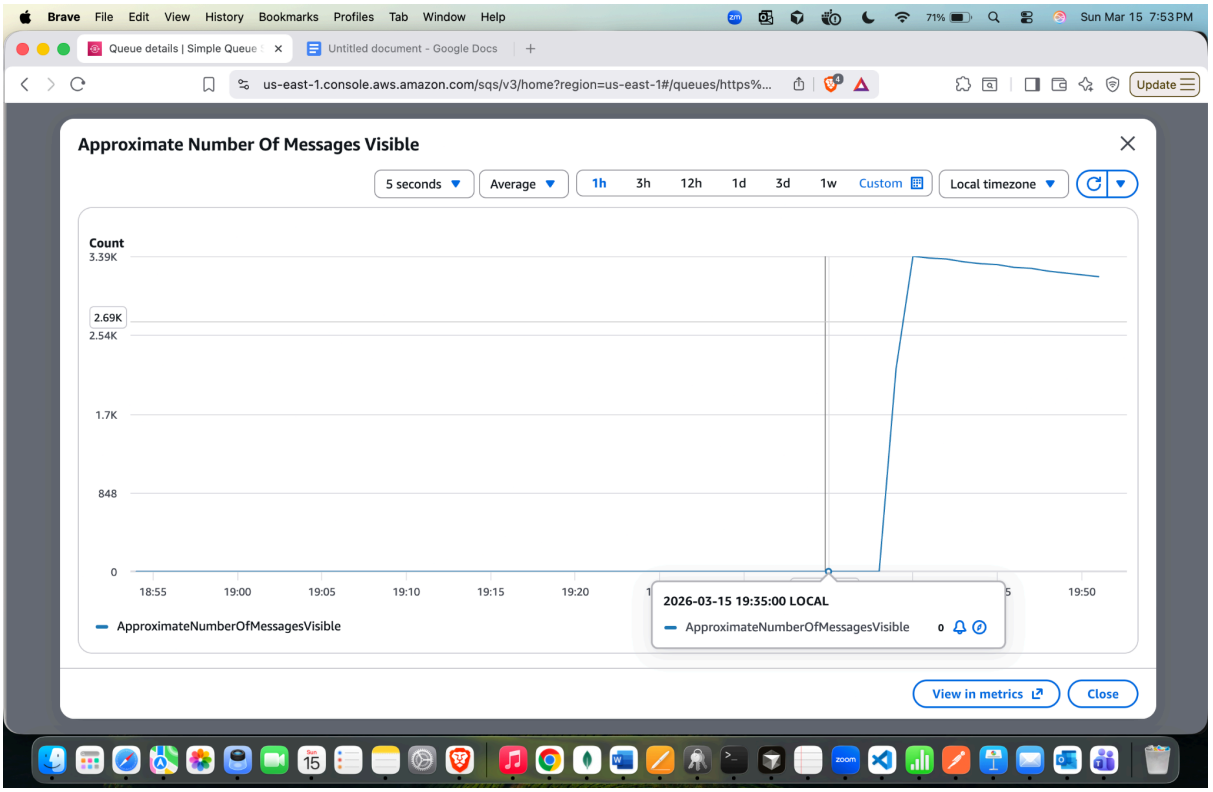
EDIT | STOP | RESET

STATISTICS | CHARTS | FAILURES | EXCEPTIONS | CURRENT RATIO | DOWNLOAD DATA | LOGS | LOCUST CLOUD

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/orders/async	3448	0	42	68	100	46.23	29	1655	214.46	59.5	0
	Aggregated	3448	0	42	68	100	46.23	29	1655	214.46	59.5	0



Phase 4: The Queue Problem

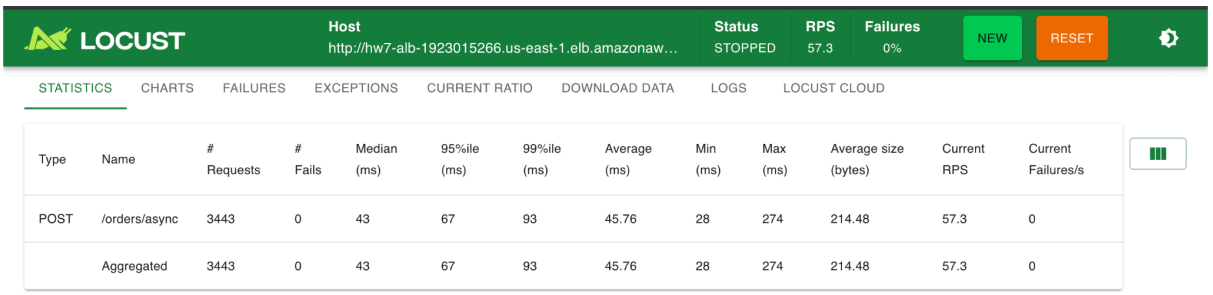


- Flash sale generated ~60 orders per second.
- Single worker processes 1 order every 3 seconds (~0.33 orders/sec).
- Orders arrive much faster than they are processed, causing a queue buildup.
- The queue grows by ~59.67 messages per second.
- CloudWatch showed ~3,400 messages in SQS waiting to be processed.
- Clearing the backlog would take 3,448 ÷ 0.33 ~10,344 seconds (~172 minutes / ~2.9 hours).
- **Result:** Large queue buildup and long delays in order confirmation for customers.

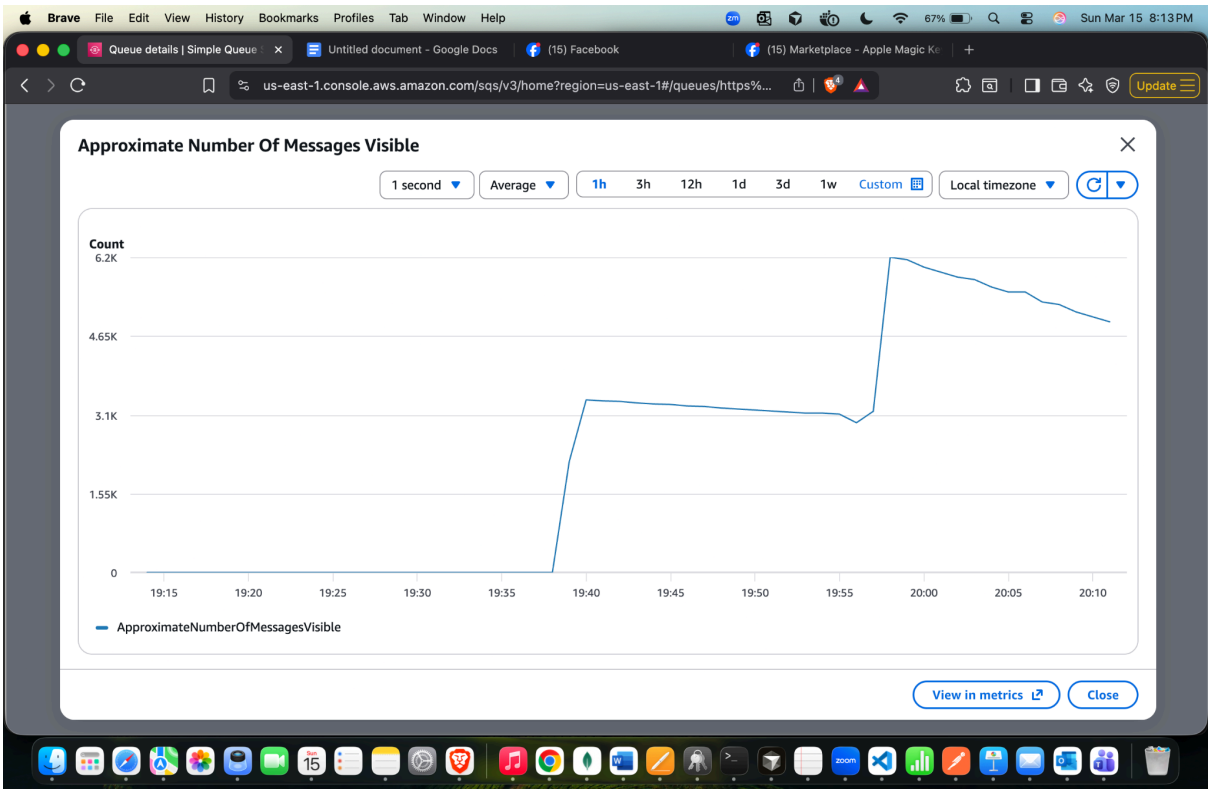
Phase 5: Scale Your Workers

Number of Workers	Processing Rate (orders/s)	Peak Queue Depth
1	0.33	~3,400
5	1.67	~3,400
20	6.67	~3,400
100	33.3	~3,400

5 workers:
Peak queue depth: ~3,400 (same intake, queue still grows fast)
Time to zero: ~34 min (calculated: 3,400 ÷ 1.67)



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/orders/async	3443	0	43	67	93	45.76	28	274	214.48	57.3	0
	Aggregated	3443	0	43	67	93	45.76	28	274	214.48	57.3	0



20 workers:

Peak queue depth: ~3,400
Time to zero: ~8.5 min (calculated: 3,400 ÷ 6.67)

LOCUST

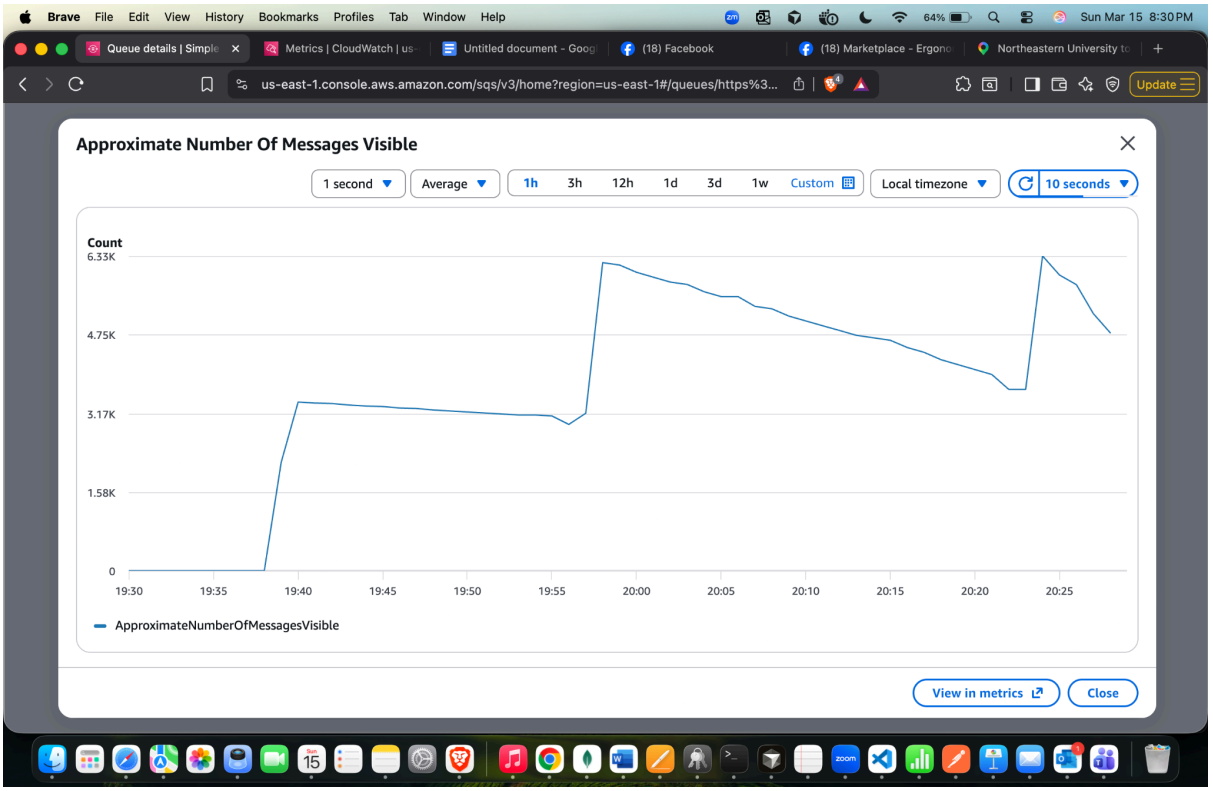
Host: http://hw7-alb-1923015266.us-east-1.elb.amazonaw…

Status: STOPPED | RPS: 58.1 | Failures: 0%

NEW | RESET

STATISTICS | CHARTS | FAILURES | EXCEPTIONS | CURRENT RATIO | DOWNLOAD DATA | LOGS | LOCUST CLOUD

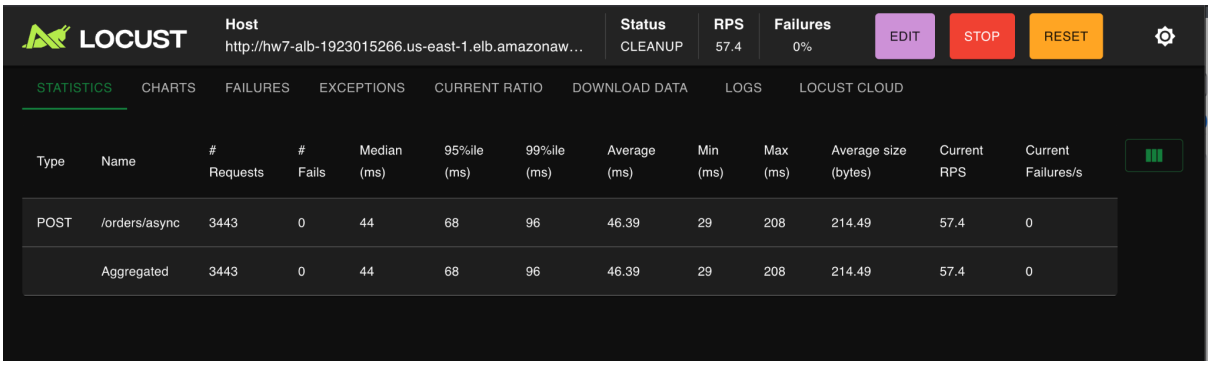
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/orders/async	3435	0	44	70	100	47.6	30	261	214.49	58.1	0
	Aggregated	3435	0	44	70	100	47.6	30	261	214.49	58.1	0

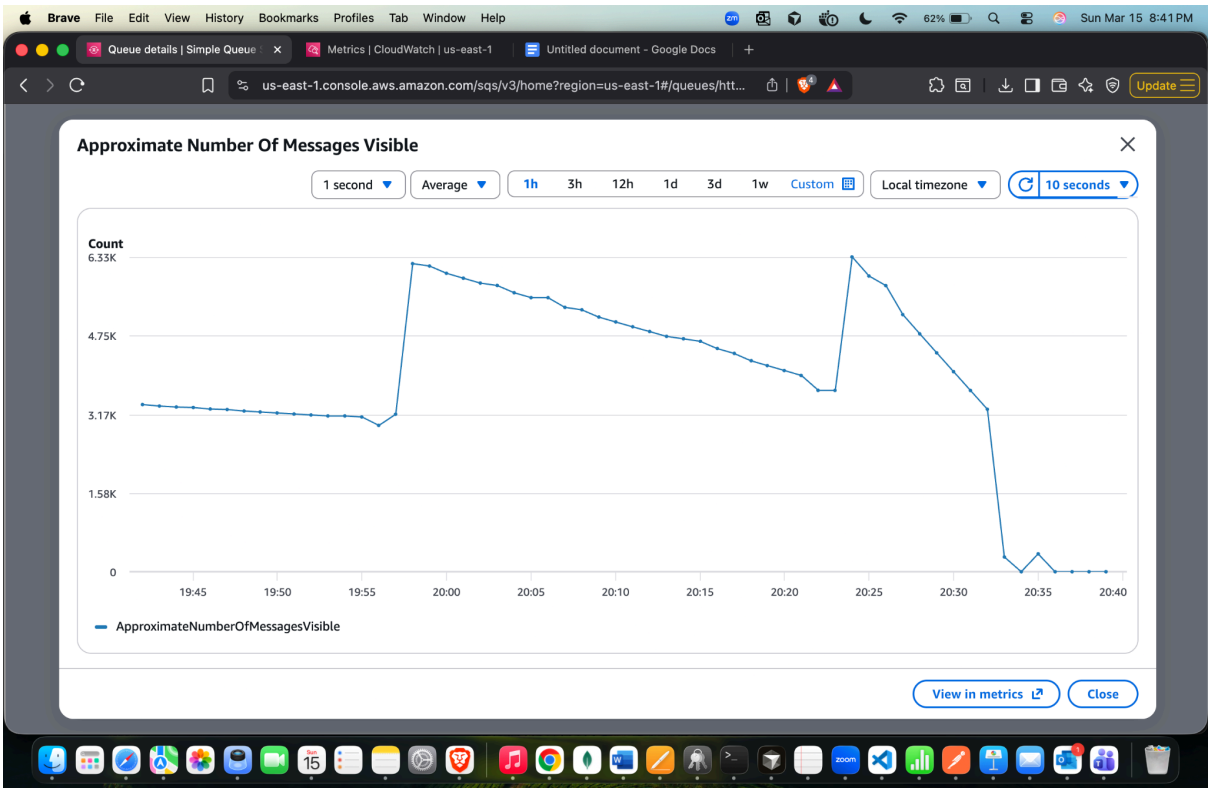


100 workers:

Peak queue depth: ~3,400

Time to zero: ~1.7 min (in CloudWatch it dropped to near 0 including all accumulated backlog)





Resource utilization:

Since the workers are I/O-bound (sleeping 3 seconds), goroutines consume almost no CPU or memory. Go goroutines start with ~8KB stack each, so even 100 goroutines use less than 1MB. The 256 CPU / 512MB ECS task handles this comfortably because sleeping goroutines don't consume CPU cycles.

Minimum workers to prevent queue buildup:

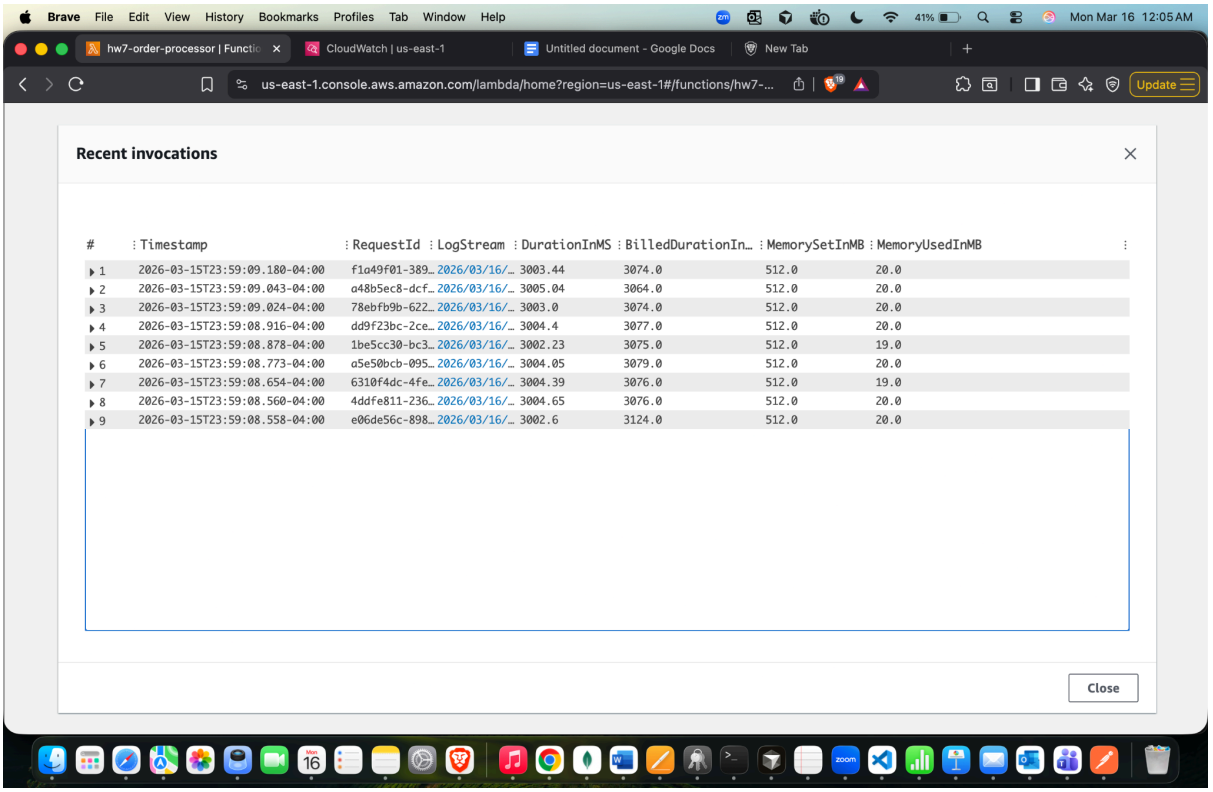
At 60 orders/second incoming, each worker processes 0.33 orders/second, so: 60 ÷ 0.33 = **180 workers** needed to match the intake rate and prevent any queue growth.

PART - 3

Deployed Lambda Function:

A Go Lambda function was deployed subscribing directly to the SNS topic `order-processing-events`, bypassing SQS entirely. Configuration: runtime `provided.al2`, 512MB memory, 10-second timeout. The function performs the same 3-second payment processing delay as the ECS workers.

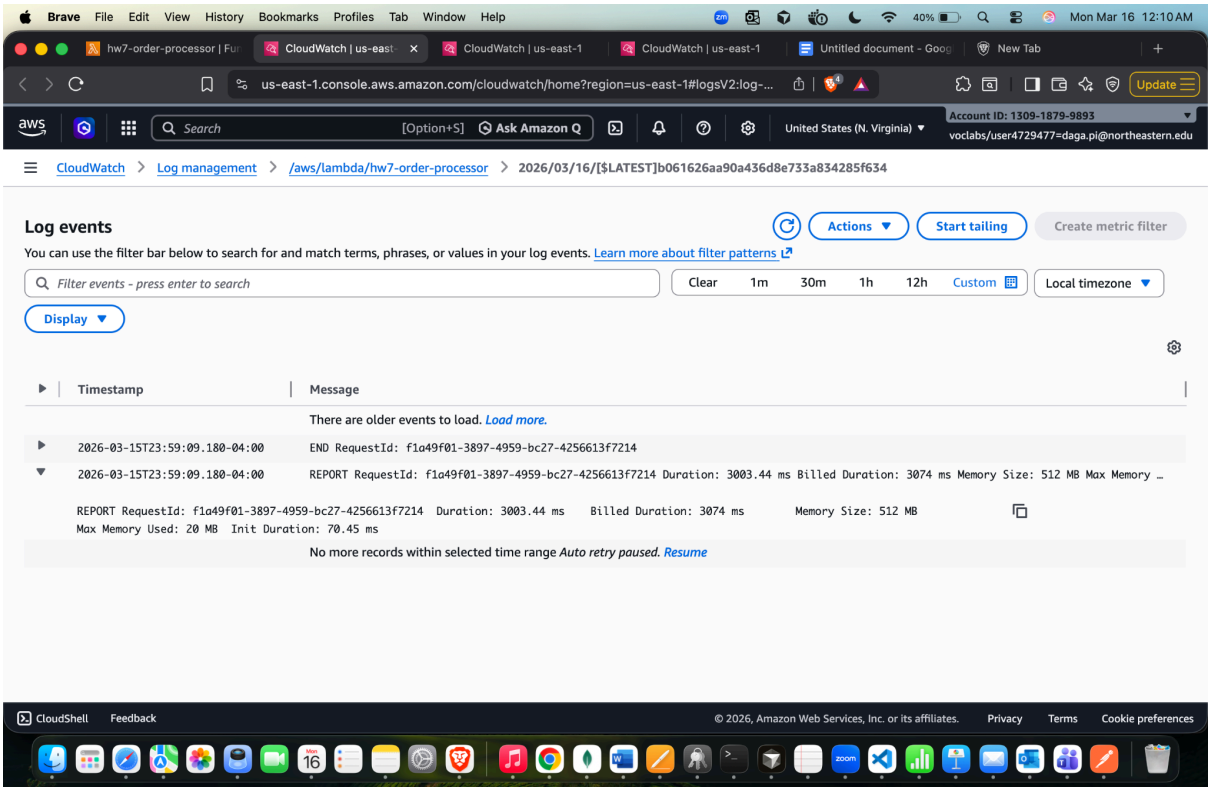
10 orders were sent via curl through the existing `/orders/async` endpoint. Lambda automatically scaled to 9 concurrent instances to handle simultaneous orders.

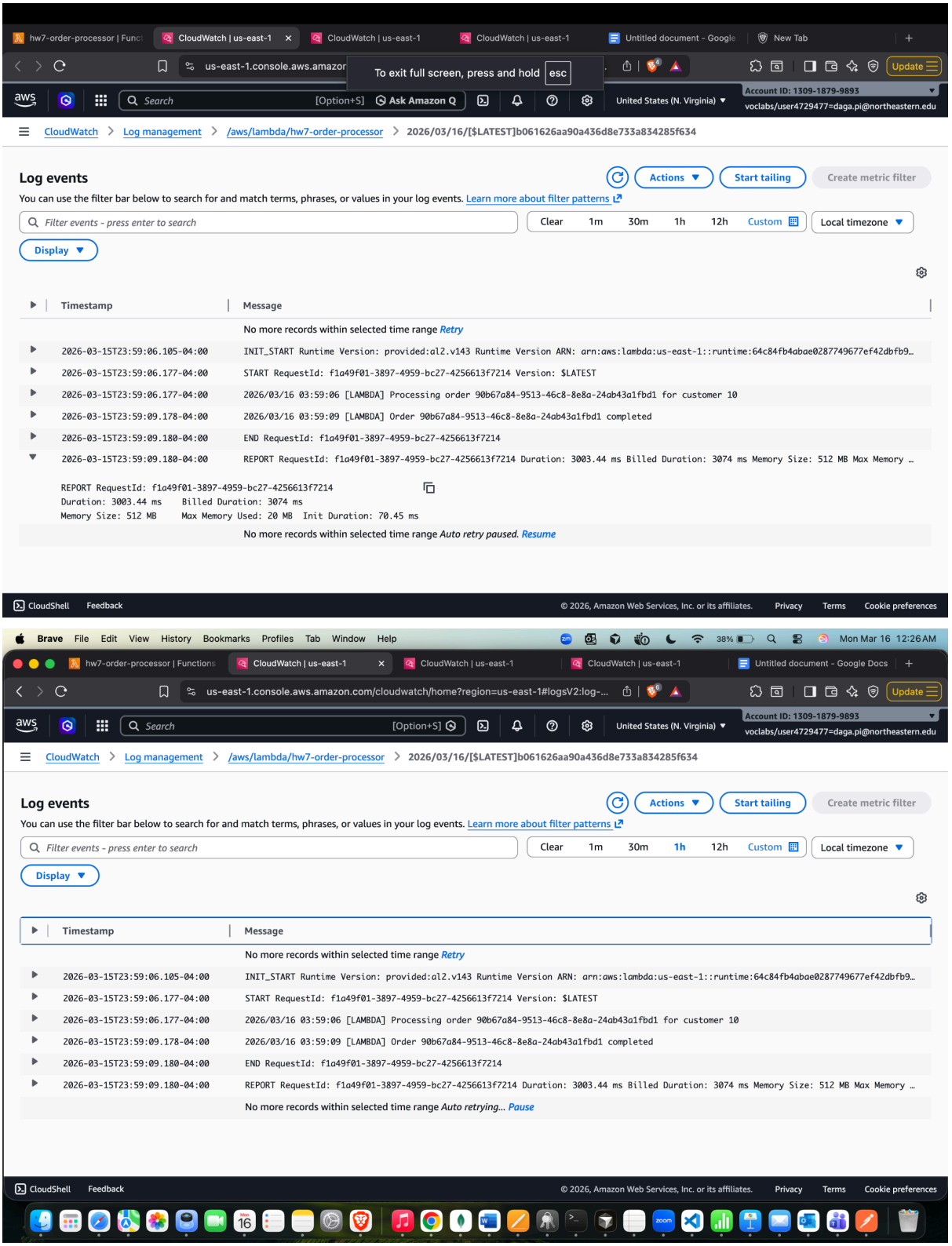


#	Timestamp	RequestId	LogStream	DurationInMS	BilledDurationIn…	MemorySetInMB	MemoryUsedInMB
1	2026-03-15T23:59:09.180-04:00	f1a49f01-389…	2026/03/16/…	3003.44	3074.0	512.0	20.0
2	2026-03-15T23:59:09.043-04:00	a48b5ec8-dcf…	2026/03/16/…	3005.04	3064.0	512.0	20.0
3	2026-03-15T23:59:09.024-04:00	78ebfb9b-622…	2026/03/16/…	3003.0	3074.0	512.0	20.0
4	2026-03-15T23:59:08.916-04:00	dd9f23bc-2ce…	2026/03/16/…	3004.4	3077.0	512.0	20.0
5	2026-03-15T23:59:08.878-04:00	1be5cc30-bc3…	2026/03/16/…	3002.23	3075.0	512.0	19.0
6	2026-03-15T23:59:08.773-04:00	a5e50bcb-095…	2026/03/16/…	3004.05	3079.0	512.0	20.0
7	2026-03-15T23:59:08.654-04:00	6310f4dc-4fe…	2026/03/16/…	3004.39	3076.0	512.0	19.0
8	2026-03-15T23:59:08.560-04:00	4ddfe811-236…	2026/03/16/…	3004.65	3076.0	512.0	20.0
9	2026-03-15T23:59:08.558-04:00	e06de56c-898…	2026/03/16/…	3002.6	3124.0	512.0	20.0

Cold Start Observations

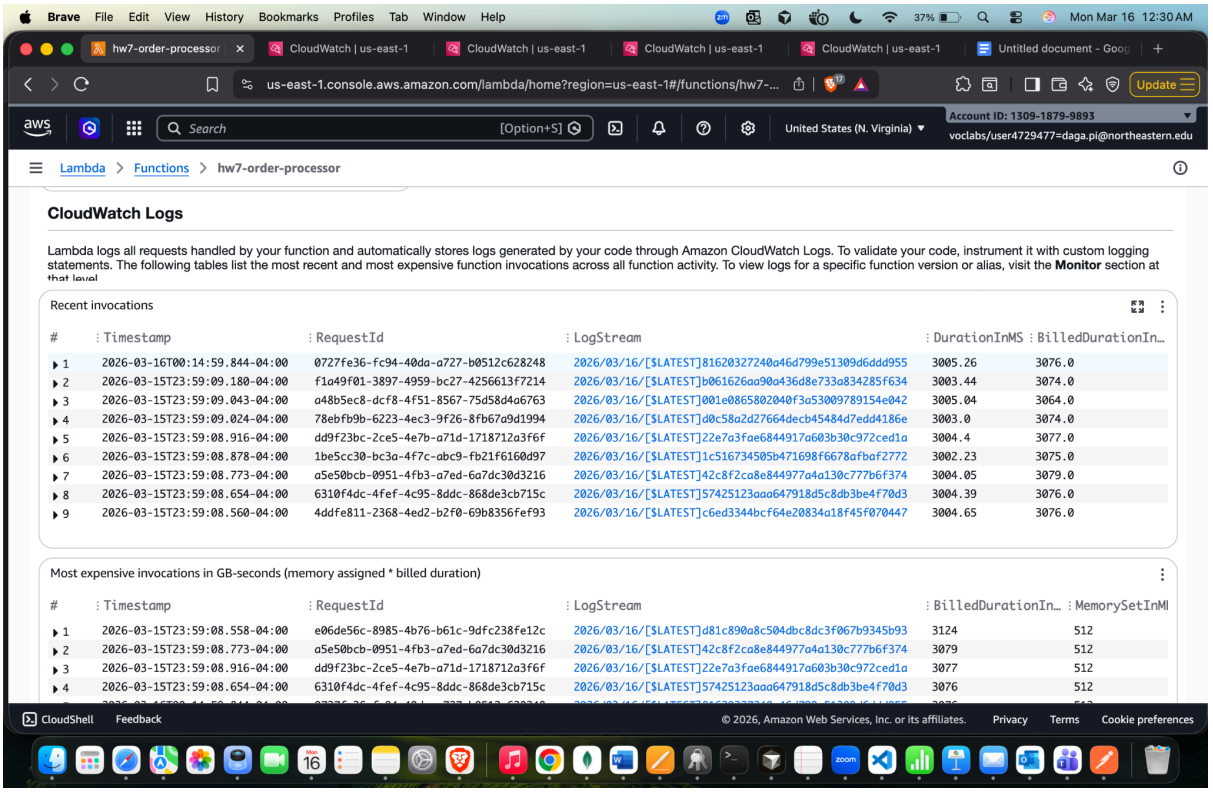
All invocations were cold starts since Lambda creates a new instance per concurrent request and recycles idle instances after ~5 minutes.

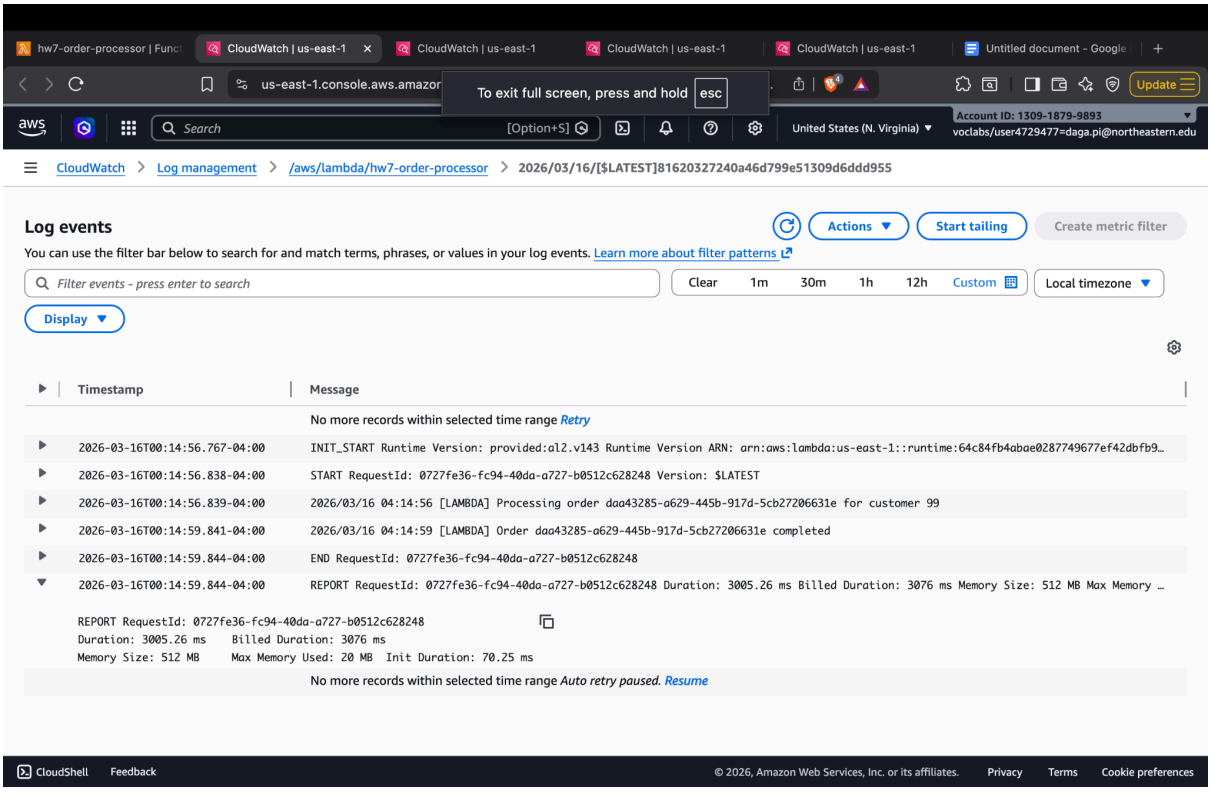

Brave File Edit View History Bookmarks Profiles Tab Window Help
Mon Mar 16 12:10 AM
hw7-order-processor | Fu
CloudWatch | us-east
CloudWatch | us-east-1
CloudWatch | us-east-1
Untitled document - Goog
New Tab
us-east-1.console.aws.amazon.com/cloudwatch/home?region=us-east-1#logsV2:log-...
Update
Search
[Option+S]
Ask Amazon Q
United States (N. Virginia)
Account ID: 1309-1879-9893
voclabs/user4729477=daga.pi@northeastern.edu
CloudWatch
Log management
/aws/lambda/hw7-order-processor
2026/03/16/[$LATEST]b061626aa90a436d8e733a834285f634
Log events
Actions
Start tailing
Create metric filter
You can use the filter bar below to search for and match terms, phrases, or values in your log events. Learn more about filter patterns
Filter events - press enter to search
Clear
1m
30m
1h
12h
Custom
Local timezone
Display
Timestamp
Message
There are older events to load. Load more.
2026-03-15T23:59:09.180-04:00
END RequestId: f1a49f01-3897-4959-bc27-4256613f7214
2026-03-15T23:59:09.180-04:00
REPORT RequestId: f1a49f01-3897-4959-bc27-4256613f7214 Duration: 3003.44 ms Billed Duration: 3074 ms Memory Size: 512 MB Max Memory …
REPORT RequestId: f1a49f01-3897-4959-bc27-4256613f7214 Duration: 3003.44 ms Billed Duration: 3074 ms Memory Size: 512 MB
Max Memory Used: 20 MB Init Duration: 70.45 ms
No more records within selected time range Auto retry paused. Resume
CloudShell
Feedback
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


hw7-order-processor | Func
CloudWatch | us-east-1
CloudWatch | us-east-1
CloudWatch | us-east-1
Untitled document - Google
New Tab
us-east-1.console.aws.amazon
To exit full screen, press and hold esc
Update
Search
[Option+S]
Ask Amazon Q
United States (N. Virginia)
Account ID: 1309-1879-9893
voclabs/user4729477=daga.pi@northeastern.edu
CloudWatch > Log management > /aws/lambda/hw7-order-processor > 2026/03/16/[$LATEST]b061626aa90a436d8e733a834285f634
Log events
Actions
Start tailing
Create metric filter
You can use the filter bar below to search for and match terms, phrases, or values in your log events. Learn more about filter patterns
Filter events - press enter to search
Clear
1m
30m
1h
12h
Custom
Local timezone
Display
Timestamp
Message
No more records within selected time range Retry
2026-03-15T23:59:06.105-04:00 INIT_START Runtime Version: provided:al2.v143 Runtime Version ARN: arn:aws:lambda:us-east-1::runtime:64c84fb4abae0287749677ef42dbfb9…
2026-03-15T23:59:06.177-04:00 START RequestId: f1a49f01-3897-4959-bc27-4256613f7214 Version: $LATEST
2026-03-15T23:59:06.177-04:00 2026/03/16 03:59:06 [LAMBDA] Processing order 90b67a84-9513-46c8-8e8a-24ab43a1fbd1 for customer 10
2026-03-15T23:59:09.178-04:00 2026/03/16 03:59:09 [LAMBDA] Order 90b67a84-9513-46c8-8e8a-24ab43a1fbd1 completed
2026-03-15T23:59:09.180-04:00 END RequestId: f1a49f01-3897-4959-bc27-4256613f7214
2026-03-15T23:59:09.180-04:00 REPORT RequestId: f1a49f01-3897-4959-bc27-4256613f7214 Duration: 3003.44 ms Billed Duration: 3074 ms Memory Size: 512 MB Max Memory …
REPORT RequestId: f1a49f01-3897-4959-bc27-4256613f7214
Duration: 3003.44 ms Billed Duration: 3074 ms
Memory Size: 512 MB Max Memory Used: 20 MB Init Duration: 70.45 ms
No more records within selected time range Auto retry paused. Resume
CloudShell
Feedback
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

Brave File Edit View History Bookmarks Profiles Tab Window Help
38%
Mon Mar 16 12:26 AM
hw7-order-processor | Functions
CloudWatch | us-east-1
CloudWatch | us-east-1
CloudWatch | us-east-1
Untitled document - Google Docs
us-east-1.console.aws.amazon.com/cloudwatch/home?region=us-east-1#logsV2:log-…
Update
Search
[Option+S]
United States (N. Virginia)
Account ID: 1309-1879-9893
voclabs/user4729477=daga.pi@northeastern.edu
CloudWatch > Log management > /aws/lambda/hw7-order-processor > 2026/03/16/[$LATEST]b061626aa90a436d8e733a834285f634
Log events
Actions
Start tailing
Create metric filter
You can use the filter bar below to search for and match terms, phrases, or values in your log events. Learn more about filter patterns
Filter events - press enter to search
Clear
1m
30m
1h
12h
Custom
Local timezone
Display
Timestamp
Message
No more records within selected time range Retry
2026-03-15T23:59:06.105-04:00 INIT_START Runtime Version: provided:al2.v143 Runtime Version ARN: arn:aws:lambda:us-east-1::runtime:64c84fb4abae0287749677ef42dbfb9…
2026-03-15T23:59:06.177-04:00 START RequestId: f1a49f01-3897-4959-bc27-4256613f7214 Version: $LATEST
2026-03-15T23:59:06.177-04:00 2026/03/16 03:59:06 [LAMBDA] Processing order 90b67a84-9513-46c8-8e8a-24ab43a1fbd1 for customer 10
2026-03-15T23:59:09.178-04:00 2026/03/16 03:59:09 [LAMBDA] Order 90b67a84-9513-46c8-8e8a-24ab43a1fbd1 completed
2026-03-15T23:59:09.180-04:00 END RequestId: f1a49f01-3897-4959-bc27-4256613f7214
2026-03-15T23:59:09.180-04:00 REPORT RequestId: f1a49f01-3897-4959-bc27-4256613f7214 Duration: 3003.44 ms Billed Duration: 3074 ms Memory Size: 512 MB Max Memory …
No more records within selected time range Auto retrying… Pause
CloudShell
Feedback
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

A second request was sent to test for warm starts.





Metric	Invocation #1	Invocation #10
Duration	3003.44 ms	3005.26 ms
Billed Duration	3074 ms	3076 ms
Init Duration	70.45 ms	70.25 ms
Memory Allocated	512 MB	512 MB
Memory Used	20 MB	20 MB

Cold start overhead: ~70ms on a 3000ms operation = **~2.3%**. This is negligible for 3-second payment processing. Cold starts occur on first invocation and after ~5 minutes of inactivity. In sustained traffic, warm starts would eliminate the ~70ms Init Duration entirely, but for this use case the impact is minimal either way.

Cost Calculation

ECS cost (Part II):

- 2 ECS tasks × $8.50/month = **$17/month** (always running, whether processing orders or not)

Lambda cost:

- $0.20 per million requests
- $0.0000166667 per GB-second
- Free tier: 1M requests + 400K GB-seconds monthly

For 10,000 orders/month at 3 seconds each, 512MB (0.5GB):

- Requests: 10,000 (under 1M free tier) = **$0**
- GB-seconds: 10,000 × 3 × 0.5 = 15,000 (under 400K free tier) = **$0**
- Monthly cost = **$0 (FREE)**

Break-even: free tier covers 400K ÷ 1.5 = **~267K orders/month**. Need ~1.7M requests/month to reach $17.

How often did cold starts occur?

Every invocation. Lambda recycled instances after ~5 minutes of idle time, so both the initial batch (9 concurrent) and the follow-up request all experienced cold starts with ~70ms Init Duration.

Is the cost advantage compelling?

Yes. Lambda costs $0 vs ECS $17/month for 10,000 orders. It remains free until 267K orders/month.

Can you accept losing SQS guarantees?

For a startup, yes. SNS retries twice then discards, whereas SQS retains for 4 days. At early stage, operational simplicity outweighs guaranteed delivery. As the business scales, SQS can be added back.

Scale consideration:

Lambda stays free until 267K orders/month, far beyond typical startup volumes.

Switch Recommendation

For processing under 267K orders/month, Lambda is is good. It eliminates $17/month in ECS costs, removes all operational overhead (no queue-depth alerts, no manual worker scaling, no ECS health monitoring), and automatically scales to handle flash sales without any configuration. The 70ms cold start overhead is negligible on 3-second payment processing (2.3% impact). The main risk is losing SQS's retry guarantees SNS only retries failed Lambda invocations twice before discarding the message, whereas SQS retains messages for up to 4 days. For operational simplicity and cost savings outweigh the need for guaranteed delivery, Lambda is the right choice. As order volume grows past 267K/month or reliability requirements tighten, adding SQS back as a buffer between SNS and Lambda (or returning to ECS workers)